

Change Target Object for Adversarial Patch

This report explains how to change the target class (for example, from person to car) when generating adversarial patches in AdvReal.

1) The single source of truth: ATTACK_CLASS

The target class is controlled by ATTACK_CLASS in your YAML config. It is parsed in ConfigParser and used by the attacker and patch placement logic.

Where it is read:

- Config parsing and target class storage: `utils/parser.py`
- Target class filtering for patch placement: `attack/attacker.py`

2) Find the class ID for "car"

The class names file is defined in the config, for example:

- COCO 80 names: `configs/namefiles/coco80.names`

To target car, you need the numeric index of "car" in that names file. In COCO80, "car" is usually index 2, but verify in the file to be safe.

3) Step-by-step (beginner friendly)

Step A: Open your config file

Pick the config you use for training (examples):

- `configs/baseline/v5.yaml`
- `configs/baseline/v11.yaml`

Step B: Set ATTACK_CLASS to the car ID

Find this block:

```
ATTACKER:
  ATTACK_CLASS: '0'
```

Change it to the car ID, for example:

```
ATTACKER:
  ATTACK_CLASS: '2'
```

Step C: Confirm the class names file

Check that DATA.CLASS_NAME_FILE in your config points to the names file you expect:

```
DATA:
  CLASS_NAME_FILE: 'configs/namefiles/coco80.names'
```

If you use a custom dataset, update this path to your own class list file.

Step D: Run training

Use your normal command, for example:

```
python train.py --nepoch 800 --save_path 'results/yolov5_car' --arch
"yolov5" --cfg configs/baseline/v5.yaml --seed_type fixed --loss_type
max_iou
```

4) Important note about train.py vs multi-detector path

There are two paths in this repo:

1. Multi-detector attacker path

- Uses UniversalAttacker and init_detectors.
- ATTACK_CLASS is respected for patch placement and filtering.

2. train.py path (3D/patch training)

- Uses model-specific probability extractors in load_data.py.
- Some extractors still hardcode attack_cls = 0 (person).

This means changing ATTACK_CLASS in the config may not fully affect the training path unless the extractor reads the config.

What to change if you use train.py

- YOLOv5MaxProbExtractor in load_data.py currently sets attack_cls = 0. Update it to read from cfg (same pattern used in YOLOv11MaxProbExtractor).

Suggested pattern:

```
attack_cls = int(getattr(self.cfg, 'ATTACKER',
EasyDict(ATTACK_CLASS='0')).ATTACK_CLASS)
mask = mask & (cls_idx == attack_cls)
```

Also ensure the extractor instance has cfg assigned after you create it.

5) Quick checklist

- Confirm the class index in configs/namefiles/coco80.names
- Update ATTACK_CLASS in the config
- If using train.py, ensure the extractor uses cfg (not hardcoded 0)
- Verify that bbox_array output uses the class ID you expect

6) Example: person to car (YOLOv11)

1. Open configs/baseline/v11.yaml
2. Change ATTACK_CLASS from '0' to '2' (if car is index 2 in your class list)
3. Run training with --arch "yolov11"

If your results still target person, update the extractor logic as described above.

7) Data and 3D asset requirements for a new target object

If you change the target class (for example, from person to car), the training data and 3D assets must also match the new object:

- Dataset: use images and labels for the new class (for example, car images instead of pedestrian).
- Class names: update the names file to include the correct class list and verify the car index.
- 3D model: provide a suitable 3D mesh for the new object (car model instead of human body).
- Rendering: update the renderer inputs (mesh paths, UVs, textures) so the new object is rendered correctly.
- Perspective and scale: verify camera distance, object scale, and placement so the patch is realistic on the new target.

If you keep person-specific data and meshes, the patch will optimize for the wrong object even if ATTACK_CLASS is changed.